INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

APLICACIONES PARA COMUNICACIONES EN RED

Reporte

“Socket bloqueante”

Grupo: 6CM2

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Moreno Cervantes Axel Ernesto

Fecha de entrega: 03 de octubre de 2025

ÍNDICE

INTRODUCCIÓN 2
DESARROLLO 4
Archivos JSON 5
1.- Archivo: Articulos.json 5
2.- Archivo: Carrito.json 6
Archivos PYTHON 6
1.- Archivo: FuncionesServidor.py 6
2.- Archivo: FuncionesCliente.py 14
3.-Archivo: ClienteTienda.py 17
4.- Archivo: ServidorTienda.py 21
Archivo .txt 23
1.- Archivo: recibo[fecha actual].txt 23
CONCLUSIÓN 24

INTRODUCCIÓN

Socket

Es el punto físico de contacto entre dos entidades que se comunican por una red. Es la abstracción que permite a programas, servicios o dispositivos intercambiar información como si tuvieran un "canal" dedicado entre ellos.

Un socket se identifica por la combinación de una dirección (por ejemplo, una dirección IP) y un número que actúa como "puerto". Eso permite que muchos servicios en la misma máquina reciban y envíen mensajes sin confundirse: cada servicio tiene su propio "extremo" o entrada.

El socket representa tanto la entrada como la salida por donde fluyen los datos; se encarga de transformar los datos del formato que usa la aplicación al formato que exige la red, y viceversa.

Tipos conceptuales de socket

- **Orientados a conexión o tipo "flujo":** la comunicación se establece de forma continuada y confiable; se garantiza que los datos lleguen en orden y sin pérdidas (o se notifican los errores).
- **Sin conexión o tipo "mensaje":** cada envío es independiente; no existe la noción de una "conversación" continua. Cada mensaje viaja por su cuenta y puede llegar en distinto orden o perderse.

Funcionalidad

- ✓ **Envío y recepción de datos:** facilita que una entidad ponga datos en la red y que otra los reciba.
- ✓ **Encaminamiento lógico:** ayuda a que los datos se entreguen al servicio correcto dentro de un equipo (gracias al puerto).
- ✓ **Manejo de condiciones de la red:** el socket abstrae fallos, latencia y límites de capacidad, exponiendo estados como "listo para enviar", "listo para recibir", "conectado" o "desconectado".

Socket bloqueante

Un socket bloqueante es aquel cuyo comportamiento hace que la entidad que intenta usarlo se detenga y espere cuando la operación requerida no puede completarse de inmediato. En términos generales:

Si se intenta leer y aún no hay datos disponibles, el socket hace que el proceso espere hasta que lleguen datos. Si se intenta escribir, pero el medio está saturado o no puede aceptar más datos en ese momento, el proceso espera hasta que haya capacidad.

Ventajas

- **Comportamiento intuitivo:** quien pide algo espera la respuesta y luego continúa.
- **Menos complejidad:** no hay que checar constantemente si ya llegó la información.

Desventajas

- **Ineficiencia si se manejan muchas comunicaciones al mismo tiempo:** muchos recursos pueden quedar "en espera".
- **Menos receptivo si la red es lenta o hay latencia**: la entidad queda inactiva durante la espera.

Módulo "socket" en Python

Módulo estándar que permite crear un punto final para la comunicación a través de una red, estableciendo conexiones entre aplicaciones, procesos o dispositivos. Se utiliza para desarrollar aplicaciones cliente-servidor y enviar/recibir datos utilizando la API de sockets de Berkeley. Para usarla, simplemente se importa el módulo socket y se utilizan funciones como socket(), bind(), listen(), accept(), connect(), send(), recv() y close() para gestionar la comunicación.

Métodos y constantes del módulo "socket"

Método	Descripción
socket()	Crea un objeto socket (el "extremo" de comunicación). Al crear uno eliges la familia (IPv4 o IPv6) y el tipo (flujo/TCP o datagrama/UDP).
bind()	Asocia el socket a una dirección local (por ejemplo, una IP y un puerto) para que pueda recibir tráfico destinado a esa dirección.
listen()	Pone un socket en modo "aceptar conexiones entrantes" (solo para sockets orientados a conexión).
accept()	Obtiene la nueva conexión entrante y devuelve un nuevo socket que representa esa conversación específica.
connect()	Establece una conexión hacia un extremo remoto (cliente).
send() / sendall()	Envía bytes a través del socket; sendall() intenta asegurarse de enviar todo el bloque hasta completarlo.
recv()	Recibe bytes del socket; devuelve lo que haya llegado (o vacío si la otra parte cerró la conexión).
close()	Cierra el socket y libera el recurso.
setblocking()	Cambia el modo de bloqueo del socket: modo bloqueante o no bloqueante.

Constantes	Descripción
AF_INET, AF_INET6	Familia de direcciones (IPv4 / IPv6).
SOCK_STREAM, SOCK_DGRAM	Tipo de socket: flujo (TCP) o datagrama (UDP).

Módulo "json" en Python

El módulo estándar json sirve para convertir entre datos de Python y el formato JSON (JavaScript Object Notation). JSON es un formato de texto muy usado para intercambio de datos entre sistemas porque es ligero, legible y portable. El módulo json es la herramienta principal en Python para serializar (de Python a JSON) y deserializar (de JSON a Python).

Mapeo básico de tipos

- **dict** ⇄ objeto JSON
- **list / tuple** ⇄ array JSON (las tuplas se convierten a arrays)
- **str** ⇄ string JSON
- **int** / float ⇄ number JSON
- **True / False** ⇄ true / false
- **None** ⇄ null

Tipos no JSON-serializables por defecto: bytes, datetime, objetos personalizados, etc.

Métodos principales

Método	Descripción
json.dumps(obj, **opts)	Convierte un objeto Python a una cadena JSON.
json.dump(obj, file, **opts)	Escribe el JSON resultante directamente en un archivo abierto.
json.loads(s, **opts)	Convierte una cadena JSON a estructuras Python.
json.load(file, **opts)	Lee JSON desde un archivo y lo convierte a Python.

DESARROLLO

Como ya se explicó en la introducción, nosotros decidimos desarrollar esta práctica usando Python y los módulos estándar “socket” y “json” en conjunto. Como resultado del desarrollo, se crearon 4 archivos Python, 2 archivos JSON y un archivo.txt. A continuación, explicaremos que hace cada uno de los archivos y las funciones que usan (en el caso de los archivos Python).

Cabe aclarar que la decisión de crear 4 archivos Python fue porque los archivos correspondientes al cliente y al servidor iban a quedar muy largos, por consecuencia más confusos de entender. Entonces se crearon dos archivos más, uno que contiene las funciones que se usan solo en el servidor y en el otro las funciones que se usan para el cliente.

Debido a que estos archivos representan toda la funcionalidad tanto del servidor como del cliente, se explicaran primero que los archivos donde se crea al mismo cliente y servidor, ya que estos últimos son más como una plantilla y por dende más cortos y fáciles de entender sabiendo de antemano para que se usan las funciones empleadas para cada uno.

Para el caso de los archivos JSON, fueron creados 2, uno donde se guardan todos los artículos disponibles en la tienda en línea, como el almacén, mientras que el otro representa el carito

de compras, en donde se guardarán los artículos de la tienda que el cliente decida agregar a este.

Y por último el archivo.txt que representa el recibo de compra, una vez que el cliente decida finalizar la compra. En este archivo se listan todos los artículos en el carrito de compras una vez finalizada la compra, así como el total a pagar, la fecha y lugar de la compra.

Para los artículos decidimos considerar la siguiente información para cada uno:

- Id: Es el id único del artículo en la tienda.
- Tipo: Es la categoría a la que pertenece el artículo (Abarrotes, Bebidas, Snacks, Cuidado personal y Limpieza).
- Nombre: Es el nombre del artículo.
- Precio: Precio del artículo por unidad.
- Marca: Marca del artículo.
- Stock: Total de unidades disponibles del artículo en la tienda.

Archivos JSON

1.- Archivo: Articulos.json

Como se explicó anteriormente, en este archivo se almacenan todos los artículos de la tienda.

```
1   {
2       "articulos": [
3           {
4               "id": 1,
5               "tipo": "Abarrotes",
6               "nombre": "Tortillas de maíz",
7               "precio": 28,
8               "marca": "Maseca",
9               "stock": 115
10          },
11          {
12              "id": 2,
13              "tipo": "Abarrotes",
14              "nombre": "Pan blanco",
15              "precio": 45,
16              "marca": "Bimbo",
17              "stock": 80
18          },
19          {
20              "id": 3,
21              "tipo": "Abarrotes",
22              "nombre": "Arroz",
23              "precio": 34,
24              "marca": "La Sierra",
25              "stock": 60
26          },
```

```
259         {
260             "id": 33,
261             "tipo": "Limpieza",
262             "nombre": "Bolsas para basura",
263             "precio": 55,
264             "marca": "BioBag",
265             "stock": 70
266         },
267         {
268             "id": 34,
269             "tipo": "Limpieza",
270             "nombre": "Esponja para platos",
271             "precio": 28,
272             "marca": "Scotch-Brite",
273             "stock": 130
274         },
275         {
276             "id": 35,
277             "tipo": "Limpieza",
278             "nombre": "Jabón para lavar platos",
279             "precio": 34,
280             "marca": "Axión",
281             "stock": 120
282         }
283     ]
284 }
```

Imagen 1. Contenido del archivo Articulos.json.

En este archivo se crea un arreglo con la clave **'articulos'**, que contiene 35 objetos que representan los 35 artículos disponibles en la tienda, cada uno con sus pares clave-valor que representan las características de cada producto y su valor. Las claves con valores enteros son el **'id'**, **'precio'** y **'stock'**, mientras que las claves con cadenas de texto son el **'tipo'**, **'nombre'** y **'marca'**.

2.- Archivo: Carrito.json

Como se explicó anteriormente, en este archivo se almacenan todos los artículos agregados al carrito de compras.

```json
{
    "carrito": [
        {
            "id": 20,
            "tipo": "Snacks",
            "nombre": "Galletas Marías",
            "precio": 28,
            "marca": "Gamesa",
            "cantidad": 2,
            "precioTotal": 56
        },
        {
            "id": 10,
            "tipo": "Bebidas",
            "nombre": "Agua de sabores",
            "precio": 22,
            "marca": "Jumex",
            "cantidad": 3,
            "precioTotal": 66
        }
    ]
}
```

Imagen 2. Contenido del archivo Carrito.json.

En este archivo se crea un arreglo con la clave **'carrito'**, que contiene n objetos que representan los n artículos agregados al carrito, cada uno con sus pares clave-valor que representan las características de cada producto y su valor. A diferencia de los artículos en el archivo **'Articulos.json'**, en este se descarta la clave **'stock'** y se agregan otras dos: **'cantidad'** para la cantidad de unidades de ese artículo y **'precioTotal'** para el precio total de las unidades agregadas de ese artículo.

Archivos PYTHON

Para explicar cada uno de los siguientes archivos y sus funciones, nos limitaremos a explicarlos en conjuntos de líneas de código y no línea por línea.

1.- Archivo: FuncionesServidor.py

Este es el archivo que contiene las funciones usadas por el servidor. A continuación, se explicará cada una de ellas.

- **obtenerRuta()**

```python
def obtenerRuta() -> tuple[str, str]:
    carpetaScript = os.path.dirname(os.path.abspath(__file__)) # Obtiene la ruta de la carpeta del script
    rutaArticulos = os.path.join(carpetaScript, "articulos.json") # Construye la ruta completa hacia "Articulos.json"
    rutaCarrito = os.path.join(carpetaScript, "Carrito.json") # Construye la ruta completa hacia "Carrito.json"

    return rutaArticulos, rutaCarrito
```

Imagen 3. Función obtenerRuta().

Descripción general: Devuelve las rutas exactas de los archivos **'Articulos.json'** y **'Carrito.json'**.

Descripción detallada: Esta función primero determina la ruta de la carpeta en donde se está ejecutando el script actual y la usa para construir la ruta de los archivos **'Articulos.json'** y **'Carrito.json'** y las devuelve.

- cargarJSON(ruta: str)

```
def cargarJSON(ruta: str) -> dict:
    with open(ruta, "r", encoding = "utf-8") as file:
        articulos = json.load(file) # Toma "file" y lo convierte a un objeto de tipo Python (deserialización)

    return articulos
```

Imagen 4. Función cargarJSON().

Descripción general: Lee el contenido del archivo **'Articulos.json'** o **'Carrito.json'** y lo convierte a un objeto de tipo Python (diccionario) y al final lo devuelve.

Parámetros:

- **ruta : str**
    - Ruta del archivo **'Articulos,json'** o **'Carrito.json'**.

Descripción detallada: Esta función toma la ruta del archivo **'Articulos.json'** o **'Carrito.json'** y la usa para abrir cualquiera de los archivos en modo lectura y el contenido lo deserializa usando la codificación utf-8 y lo devuelve como un diccionario.

- **guardarJSON(ruta: str, diccionarioNuevo: dict)**

```
def guardarJSON(ruta: str, diccionarioNuevo: dict) -> None:
    with open(ruta, "w", encoding = "utf-8") as file:
        json.dump(diccionarioNuevo, file, indent = 4, ensure_ascii = False)
```

Imagen 5. Función guardarJSON().

Descripción general: Actualiza el contenido del archivo **'Articulos.json'** o **'Carrito.json'**.

Parámetros:

- **ruta : str**
    - Ruta del archivo **'Articulos.json'** o **'Carrito.json'**.
- **diccionarioNuevo : dict**
    - Diccionario con las nuevas modificaciones a cargar.

Descripción detallada: Esta función toma la ruta del archivo **'Articulos,json'** y **'Carrito.json'** y la usa para abrir cualquiera de los archivos en modo escritura y actualiza su contenido con el diccionario nuevo serializado usando la codificación utf-8.

- **enviarMensaje(mensaje: str, conexion: socket.sokect)**

```python
def enviarMensaje(mensaje: str, conexion: socket.socket) -> None:
    mensajeEnviar = {"mensaje": [{"msj": f"{mensaje}"}]} # Se crea el JSON con el mensaje

    print(f"\n>> Se envia al cliente: {mensajeEnviar}") # Se imprime lo que se envía al cliente

    respuesta = json.dumps(mensajeEnviar).encode("utf-8") # Convierte el diccionario a una caden
    conexion.send(respuesta) # Envía los bytes al cliente a traves del socket "conexion"
```

Imagen 6. Función enviarMensaje().

Descripción general: Envía mensajes al cliente desde el servidor.

Parámetros:

- **mensaje : str**
  - Mensaje para enviar al cliente desde el servidor.
- **conexion : socket.socket**
  - Nuevo socket que representa la conexión con un cliente en particular.

Descripción detallada: Esta función crea un objeto JSON con el mensaje como el único objeto del arreglo con la clave **'mensaje'**. Este se serializa usando la codificación utf-8 y se envía al cliente usando la conexión con el cliente.

- **enviarArticulos(ruta: str, conexion: socket.socket)**

```python
def enviarArticulos(ruta: str, conexion: socket.socket) -> None:
    articulos = cargarJSON(ruta) # Obtenemos el diccionario con los artículos

    # Verificamos si hay artículos en la tienda o en el carrito
    if ("articulos" in articulos):
        if (not articulos["articulos"]): # Si no hay artículos en la tienda
            enviarMensaje("Servidor: No hay artículos para mostrar.", conexion)
        else:
            print(f"\n>> Se envia al cliente: {articulos}") # Se imprime lo que s

            respuesta = json.dumps(articulos).encode("utf-8") # Convierte el dicc
            conexion.send(respuesta) # Envía los bytes al cliente a traves del so

    if ("carrito" in articulos):
        if (not articulos["carrito"]): # Si no hay artículos en el carrito
            enviarMensaje("Servidor: No hay artículos para mostrar.", conexion)
        else:
            print(f"\n>> Se envia al cliente: {articulos}") # Se imprime lo que s

            respuesta = json.dumps(articulos).encode("utf-8") # Convierte el dicc
            conexion.send(respuesta) # Envía los bytes al cliente a traves del so
```

Imagen 7. Función enviarArticulos().

Descripción general: Esta función envía los artículos de la tienda o del carrito al cliente (si existen), de lo contrario se envía un mensaje.

Parámetros:

- **ruta : str**
  - Ruta del archivo **'Articulos.json'** o **'Carrito.json'**.
- **conexion : socket.socket**
  - Nuevo socket que representa la conexión con un cliente en particular.

Descripción detallada: Esta función toma la ruta del archivo **'Articulos.json'** o **'Carrito.json'**, para obtener el diccionario correspondiente. Se identifica si los artículos son de la tienda o del carrito usando la clave correspondiente de cada uno. Una vez hecho lo anterior, el diccionario se envía al cliente.

- **buscarArticulo(rutaArticulos: str, criterioBusqueda: str, conexion: socket.socket)**

```python
def buscarArticulo(rutaArticulos: str, criterioBusqueda: str, conexion: socket.socket) -> None:
    buscar = criterioBusqueda.lower() # Guardamos el nombre o marca del artículo
    articulos = cargarJSON(rutaArticulos) # Guardamos los artículos en un diccionario
    counter = 0 # Contador de artículos encontrados

    coincidencias = {"articulos": []} # Cremos el JSON con un diccionario vacío

    for art in articulos.get("articulos", []): # Buscamos los artículos
        if ((buscar in art["nombre"].lower()) or (buscar in art["marca"].lower())):
            coincidencias["articulos"].append(art) # Agregamos el artículo que coindice con la búsqueda
            counter += 1

    if (counter > 0): # Si se encontró al menos un artículo
        print(f"\n>> Se envia al cliente: {coincidencias}") # Se imprime lo que se envía al cliente

        respuesta = json.dumps(coincidencias).encode("utf-8") # Serializar el JSON
        conexion.send(respuesta)
    else: # Si no se encontro ningún artículo
        enviarMensaje("Servidor: No hay coincidencias.", conexion)
```

Imagen 8. Función buscarArticulo().

Descripción general: Busca un artículo en la tienda de acuerdo con el nombre o marca indicada por el cliente.

Parámetros:

- **rutaArticulos : str**
  - Ruta del archivo **'Articulos.json'**.
- **criterioBusqueda : str**
  - Marca o nombre del artículo a buscar.
- **conexion : socket.socket**
  - Nuevo socket que representa la conexión con un cliente en particular.

Descripción detallada: Esta función toma la ruta del archivo **'Articulos.json'** para obtener el diccionario. Guarda la marca o el nombre del artículo.

Crea un arreglo vacío con la clave **'articulos'**, que se usa para guardar cada uno de los artículos que coinciden con la marca o nombre al recorrer el diccionario obtenido. Por cada artículo encontrado se aumenta un contador, si este es mayor a 0 entonces hubo resultados y

se envía al cliente el arreglo creado con los artículos encontrados, de lo contrario se envía el mensaje correspondiente.

- **agregarCarrito(rutaArticulos: str, rutaCarrito: str, criterioBusqueda: str, cantidad: int, conexion: socket.socket)**

```python
def agregarCarrito(rutaArticulos: str, rutaCarrito: str, criterioBusqueda: str, cantidad: int, conexion: socket.socket) -> None:
    # Revisamos si el criterio de búsqueda fue el nombre o el id del artículo y ajustamos
    # el tipo de la varibale 'buscar'
    if (criterioBusqueda.isdigit()):
        buscar = int(criterioBusqueda)
    else:
        buscar = criterioBusqueda.lower()

    articulos = cargarJSON(rutaArticulos) # Guardamos los artículos en un diccionario
    carrito = cargarJSON(rutaCarrito) # Guardamos los artículos del carrito en un diccionario

    articuloAgregar = None # Aquí se guarda el artículo a agregar

    # Buscamos el artículo a agregar
    for art in articulos.get("articulos", []):
        if (isinstance(buscar, int) and buscar == art["id"]):
            articuloAgregar = art

            break
        elif (isinstance(buscar, str) and buscar in art["nombre"].lower()):
            articuloAgregar = art

            break

    # Verificamos que se haya encontrado el artículo
    if (not articuloAgregar):
        enviarMensaje("Servidor: El artículo no existe en la tienda.", conexion)

        return
```

```python
    # Verificamos si el stock del artículo es suficiente
    if (cantidad > articuloAgregar["stock"]):
        enviarMensaje(f"Servidor: No hay suficiente stock para agregar {cantidad} artículos.", conexion)

        return

    # Verificamos si el artículo encontrado ya esta en el artículo
    for item in carrito["carrito"]:
        if (item["id"] == articuloAgregar["id"]): # Si ya esta el artículo en el carrito
            if (item["cantidad"] + cantidad > 5): # Si es menor qur 5
                enviarMensaje("Servidor: No pudes agregar más de cinco unidades del mismo artículo.", conexion)

                return

            item["cantidad"] += cantidad # Actualizamos la cantidad del artículo en el carrito
            item["precioTotal"] = item["precio"] * item["cantidad"] # Actualizamos el precio total del artículo

            guardarJSON(rutaCarrito, carrito) # Actualizamos el artículo en el carrito
            enviarMensaje("Servidor: El artículo se actualizó en el carrito.", conexion)

            return

    # Se crea el artículo que se va a agregar al carrito, si este no existe previamente
    itemCarrito = articuloAgregar.copy()
    itemCarrito.pop("stock", None) # Se saca el item none del artículo a agregar
    itemCarrito["cantidad"] = cantidad # Se crea el item 'cantidad' en el artículo a agregar
    itemCarrito["precioTotal"] = itemCarrito["precio"] * cantidad # Se calcula el precio total y se cre el item

    carrito["carrito"].append(itemCarrito) # Se agregan los items creados al artículo a agregars
    guardarJSON(rutaCarrito, carrito) # Se agrega el artículo al carrito
    enviarMensaje("El artículo se agregó al carrito", conexion)
```

Imagen 9. Función agregarCarrito().

Descripción general: Agrega un artículo al carrito de compras.

Parámetros:

- **rutaArticulos : str**
  - Ruta del archivo **'Articulos.json'**.
- **rutaCarrito : str**
  - Ruta del archivo **'Carrito.json'**.
- **criterioBusqueda : str**
  - El id o el nombre del artículo a agregar.
- **conexion : socket.socket**
  - Nuevo socket que representa la conexión con un cliente en particular.

Descripción detallada: Se crea la variable **'buscar'** y se ajusta su tipo de acuerdo si el **'criterioBusqueda'** corresponde al id o nombre del artículo. Se obtienen los diccionarios de los archivos **'Articulos.json'** y **'Carrito.json'**. Y por último se crea un objeto, en donde se guarda el artículo a agregar.

Se recorre el diccionario de **'Articulos.json'** para buscar aquel que corresponda con el id o nombre de este. Si hay una coincidencia, el artículo se guarda. Para agregarlo al carrito se verifica si este ya existe, de ser así solo se aumenta la cantidad de unidades y se actualiza **'Carrito.json'**, de lo contrario se agrega el artículo al carrito y se actualiza **'Carrito.json'**. En cualquiera de los dos casos, se envía un mensaje de confirmación.

- **eliminarCarrito(rutaCarrito: str, criterioBusqueda: str, cantidad: int, conexion: socket.socket)**

```python
def eliminarCarrito(rutaCarrito: str, criterioBusqueda: str, cantidad: int, conexion: socket.socket) -> None:
    # Revisamos si el criterio de búsqueda fue el nombre o el id del artículo y ajustamos
    # el tipo de la varibale 'buscar'
    if (criterioBusqueda.isdigit()):
        buscar = int(criterioBusqueda)
    else:
        buscar = criterioBusqueda.lower()

    carrito = cargarJSON(rutaCarrito) # Guardamos los artículos del carrito en un diccionario

    articuloEliminar = None # Aquí se guarda el artículo a eliminar

    # Buscamos el artículo a eliminar
    for art in carrito.get("carrito", []):
        if (isinstance(buscar, int) and buscar == art["id"]):
            articuloEliminar = art

            break
        elif (isinstance(buscar, str) and buscar in art["nombre"].lower()):
            articuloEliminar = art

            break

    # Verificamos que se haya encontrado el artículo
    if (not articuloEliminar):
        enviarMensaje("Servidor: El artículo no existe en el carrito.", conexion)

        return

    # Verificamos si la cantidad del artículo es suficiente
    if (cantidad > articuloEliminar["cantidad"]):
        enviarMensaje(f"Servidor: No hay suficiente artículos para eliminar {cantidad} artículos.", conexion)

        return
```

```
    # Verificamos si el artículo encontrado ya esta en el carrito
    for item in carrito["carrito"]:
        if (item["id"] == articuloEliminar["id"]): # Si ya esta el artículo en el carrito
            if ((item["cantidad"] - cantidad) > 0): # Si todavía restan artículos en el carrito
                item["cantidad"] -= cantidad # Actualizamos la cantidad del artículo en el carrito
                item["precioTotal"] = item["precio"] * item["cantidad"] # Actualizamos el precio total del artículo

                guardarJSON(rutaCarrito, carrito) # Actualizamos el artículo en el carrito
                enviarMensaje("Servidor: El artículo se actualizó en el carrito.", conexion)
            else: # Ya no hay artículos
                carrito["carrito"].remove(item) # Eliminamos el artículo del carrito

                guardarJSON(rutaCarrito, carrito) # Actualizamos el artículo en el carrito
                enviarMensaje("Servidor: El artículo se eliminó del carrito.", conexion)

            return
```

Imagen 10. Función eliminarCarrito().

Descripción general: Elimina un artículo del carrito de compras.

Parámetros:

- **rutaCarrito : str**
  - Ruta del archivo **'Carrito.json'**.
- **criterioBusqueda : str**
  - El id o el nombre de artículos a eliminar.
- **cantidad : int**
  - Cantidad de unidades de ese artículo a eliminar.
- **conexion : socket.socket**
  - Nuevo socket que representa la conexión con un cliente en particular.

Descripción detallada: Se crea la variable **'buscar'** y se ajusta su tipo de acuerdo si el **'criterioBusqueda'** corresponde al id o nombre del artículo. Se obtiene el diccionario del archivo **'Carrito.json'**. Y por último se crea un objeto, en donde se guarda el artículo a eliminar.

Se recorre el diccionario de **'Carrito.json'** para buscar aquel que corresponda con el id o nombre de este. En el caso en el que el artículo a eliminar no se encuentra o la cantidad a eliminar sea mayor a la que hay en el carrito, el servidor envía el mensaje correspondiente.

Para eliminarlo del carrito se verifica si este ya se encuentra en este, de ser así solo se decrementa la cantidad de unidades (si hay las suficientes) y se actualiza **'Carrito.json'**. Si al eliminar artículos, la cantidad de este quede en ceros, entonces se elimina el artículo completo del carrito y se envía al cliente el mensaje de confirmación

- **finalizarCompra(rutaArticulos: str, rutaCarrito: str, conexion: socket.socket)**

```python
def finalizarCompra(rutaArticulos: str, rutaCarrito: str, conexion: socket.socket) -> None:
    articulos = cargarJSON(rutaArticulos) # Guardamos los artículos en un diccionario
    carrito = cargarJSON(rutaCarrito) # Guardamos los artículos del carrito en un diccionario
    fecha = datetime.now().strftime("%Y-%m-%d") # Se obtiene la fecha actual
    nombreArchivo = f"recibo_{fecha}.txt" # Se crea el nombre del recibo

    totalPagar = 0

    # Verificamos que haya algo en el carrito
    if ("carrito" in carrito):
        if (not carrito["carrito"]): # Si no hay artículos en el carrito
            enviarMensaje("Servidor: La compra no puede proceder, el carrito esta vacío.", conexion)
        else:
            # Primero creamos el recibo de la compra
            with open(nombreArchivo, "w", encoding = "utf-8") as file:
                file.write("************ RECIBO DE COMPRA ************\n\n")
                file.write(f"Escuela Superior de Cómputo a {fecha}\n\n")
                for itemC in carrito["carrito"]:
                    file.write(
                        f"{itemC['nombre']} ({itemC['marca']}:)\n"
                        f"- Cantidad: {itemC['cantidad']}\n"
                        f"- Precio unitario: ${itemC['precio']} MXN\n"
                        f"- Subtotal: ${itemC['precioTotal']} MXN\n\n"
                    )

                    totalPagar += itemC["precioTotal"]

                file.write("==========================================\n")
                file.write(f"TOTAL A PAGAR: ${totalPagar} MXN\n")
                file.write("==========================================\n")
```

```python
            # Actualizamos el stock de los artículos
            for itemC in carrito["carrito"]: # Iteramos el carrito
                for itemA in articulos["articulos"]: # Iteramos los artículos
                    if (itemC["id"] == itemA["id"]):
                        itemA["stock"] -= itemC["cantidad"] # Restamos la cantidad del artículo al stock de este

                        guardarJSON(rutaArticulos, articulos)

            carrito["carrito"].clear() # Borramos los artículos del carrito
            guardarJSON(rutaCarrito, carrito)

            enviarMensaje(f"Servidor: El total a pagar de los artículos del carrito es ${totalPagar} MXN. Consulta el recibo de tu compra.", conexion)
```

Imagen 11. Función finalizarCompra().

Descripción general: Finaliza la compra de los artículos en el carrito.

Parámetros:

- **rutaArticulos : str**
  - Ruta del archivo **'Artiuclos.json'**.
- **rutaCarrito : str**
  - Ruta del archivo **'Carrito.json'**.
- **conexion : str**
  - Nuevo socket que representa la conexión con un cliente en particular.

Descripción detallada: Se obtiene los diccionarios correspondientes a los archivos **'Articulos.json'** y **'Carrito.json'**, así como la fecha actual, para usarla en el nombre del recibo de la compra.

Para finalizar la compra, se verifica que haya al menos un artículo en el carrito, de lo contrario se envía al cliente el mensaje correspondiente. Si no es así, se crea un archivo.txt y se abre

en modo escritura, y entonces a la vez que se recorre el diccionario del carrito, cada artículo se escribe en el archivo y el precio total se calcula, para que al final del recorrido se escriba también.

Por último, solo queda actualizar el **'stock'** de los artículos comprados en el archivo **'Articulos.json'**, para eso recorremos ambos diccionarios y guardamos los cambios. Y al final se vacía el contenido del carrito, enviándole un mensaje al cliente de confirmación con el total a pagar e indicándole que el recibo está listo.

2.- Archivo: FuncionesCliente.py

Este es el archivo que contiene las funciones usadas por el cliente. A continuación, se explicará cada una de ellas.

- **listarTipo(articulos: dict, tipoArticulo: str)**

```python
def listarTipo(articulos: dict, tipoArticulo: str) -> None:
    for item in articulos["articulos"]:
        if (item['tipo'] == tipoArticulo): # Si coincide con el tipo del artículo
            print(f"\nId: {item['id']}")
            print(f"Nombre: {item['nombre']}")
            print(f"Precio: ${item['precio']} MXN")
            print(f"Marca: {item['marca']}")
            print(f"Stock: {item['stock']} artículos disponibles")
```

Imagen 12. Función listarTipo().

Descripción general: Esta función recibe el diccionario con los artículos de la tienda, lo recorre y los imprime separándolos por categoría.

Parámetros:

- **artículos : dict**
  - Diccionario con los artículos.
- **tipoArticulo : str**
  - Uno de los tipos de los artículos: "Abarrotes", "Bebidas", "Snacks", "Cuidado personal" y "Limpieza".

Descripción detallada: La función recibe el diccionario y el tipo del artículo a imprimir. Recorre el diccionario para obtener cada uno de los artículos, compara su tipo con la recibida y si coinciden se imprime el artículo con todas sus características.

- **listarArticulos(articulos: dict)**

```python
def listarArticulos(articulos: dict) -> None:
    print("Lista de artículos de la tienda:\n")

    print("__Artículos de tipo Abarrotes__")
    listarTipo(articulos, "Abarrotes")
    print("\n__Artículos de tipo Bebidas__")
    listarTipo(articulos, "Bebidas")
    print("\n__Artículos de tipo Snacks__")
    listarTipo(articulos, "Snacks")
    print("\n__Artículos de tipo Cuidado personal__")
    listarTipo(articulos, "Cuidado personal")
    print("\n__Artículos de tipo Limpieza__")
    listarTipo(articulos, "Limpieza")
```

Imagen 13. Función listarArticulos().

Descripción general: Esta función imprime todos los artículos de la tienda.

Parámetros:

- **artículos : dict**
    - Diccionario con los artículos.

Descripción detallada: Esta función llama varias veces a la función **listarTipo()**, a la que manda el diccionario con los artículos y el tipo de artículos a imprimir.

- **limpiarTerminal()**

```python
def limpiarTerminal() -> None:
    os.system('cls' if os .name == 'nt' else 'clear')
```

Imagen 14. Función limpiarTerminal().

Descripción general: Como lo dice su nombre, esta función se encarga de limpiar la terminal cada que se termina una acción en el cliente.

- **mostrarBusqueda(búsqueda: dict)**

```python
def mostrarBusqueda(busqueda: dict) -> None:
    print("\nArtículo(s) encontrado(s):")

    for item in busqueda["articulos"]:
        print(f"\nId: {item['id']}")
        print(f"Tipo: {item['tipo']}")
        print(f"Nombre: {item['nombre']}")
        print(f"Precio: ${item['precio']} MXN")
        print(f"Marca: {item['marca']}")
        print(f"Stock: {item['stock']}")
```

Imagen 15. Función mostrarBusqueda().

Descripción general: Esta función imprime todos los artículos que coinciden con la búsqueda.

Parámetros:

- **busqueda : dict**
  - Diccionario con los artículos encontrados en la búsqueda.

Descripción detallada: La función recibe el diccionario con los artículos encontrados, lo recorre y los imprime junto con todas sus características.

- **mostrarMensaje(mensaje: dict)**

```
def mostrarMensaje(mensaje: dict) -> None:
    for msj in mensaje["mensaje"]:
        print(f">> {msj['msj']}")
```

Imagen 16. Función mostrarMensaje().

Descripción general: Muestra los mensajes enviados por el servidor al cliente.

Parámetros:

- **mensaje : dict**
  - Diccionario de un solo elemento con el mensaje.

Descripción detallada: La función recibe el diccionario del mensaje, imprime el valor de la clave **'msj'**.

- **mostrarCarrito(carrito: dict)**

```
def mostrarCarrito(carrito: dict) -> None:
    print("Articulos en el carrito:")

    for item in carrito["carrito"]:
        print(f"\nId: {item['id']}")
        print(f"Tipo: {item['tipo']}")
        print(f"Nombre: {item['nombre']}")
        print(f"Precio: ${item['precio']} MXN")
        print(f"Marca: {item['marca']}")
        print(f"Cantidad: {item['cantidad']}")
        print(f"Precio total: ${item['precioTotal']} MXN")
```

Imagen 17. Función mostrarCarrito().

Descripción general: Imprime todos los artículos del carrito de compras.

Parámetros:

- **carrito : dict**
  - Diccionario con los artículos del carrito de compras.

Descripción detallada: Esta función recibe el diccionario con los artículos del carrito de compras actual, lo recorre e imprime cada articulo con sus características.

- **esperarTecla()**

```
120 def esperarTecla():
121     return msvcrt.getch().decode("utf-8")  # devuelve la tecla como string
```

Imagen 18. Función esperarTecla().

Descripción general: Esta función espera a que se presione una tecla cualquiera y devuelve la tecla como string.

3.-Archivo: ClienteTienda.py

```
14 import json, socket
15 from FuncionesCliente import listarArticulos, limpiarTerminal, mostrarBusqueda
16 from FuncionesCliente import mostrarMensaje, mostrarCarrito, esperarTecla
17
18 # Cremos al cliente y lo conectamos al servidor
19 cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Creamos al cliente (dirección IPv4, protocolo TCP)
20 cliente.connect(("127.0.0.1", 5000)) # Conectamos al cliente con el servidor (dirección IP, puerto)
```

Imagen 19. Creación del socket del cliente.

Líneas [14-16]: Se importan los módulos y las funciones del archivo **'FuncionesCliente'** necesarias.

Líneas [19-20]: Se crea el socket del cliente, indicando que vamos a usar direcciones IPv4 y el protocolo TCP. Después conectamos el socket a la dirección **"127.0.0.1"** y al puerto **5000**.

```
22 # Menú del cliente
23 while (True): # Mientras no se cierre el cliente
24     limpiarTerminal()
25     print("""
26 /*-------------------------.
27 | TIENDA EN LÍNEA: LALITOXDE |
28 `-------------------------*/
29
30 >> Elije una de las opciones
31
32 1.- Listar artículos de la tienda
33 2.- Buscar un artículo
34 3.- Carrito de compras
35 4.- Finalizar la compra
36 5.- Salir de la tienda
37 """)
38
39     opcion = input("Opción: ").strip()
40
41     match (opcion):
```

Imagen 20. Menú principal del cliente.

Línea [23]: Se crea un bucle while, para que el cliente pueda estar enviando solicitudes al cliente sin cerrarse la conexión a este.

Línea [25-41]: Es el menú principal de la tienda donde se muestran las diferentes opciones. El cliente al elegir una opción y usando un match(), le redirige al apartado donde se ejecuta esa opción.

```
42          case '1':
43              print("""
44  /*-------------------------------.
45  | LISTA DE ARTÍCULOS EN LA TIENDA |
46  `--------------------------------*/
47  """)
48              solicitud = {"accion": "LISTAR_ARTICULOS"} # Creamos la solicitud como un diccionario
49              cliente.send(json.dumps(solicitud).encode("utf-8")) # Enviamos al servidor la solicitud serializada
50              datosRecibidos = cliente.recv(4096).decode() # Recibe y deserializa los datos recibidos desde el servidor
51
52              articulos = json.loads(datosRecibidos) # Convierte los datos recibidos en JSON a un diccionario
53
54              if ("mensaje" in articulos): # No hay artículos en la tienda
55                  mostrarMensaje(articulos)
56              else: # Si hay al menos un artículos en la tienda
57                  listarArticulos(articulos)
58
59              # Esperamos una tecla
60              print("\n>> Presiona una tecla para continuar...")
61              tecla = esperarTecla()
```

Imagen 21. Opción listar artículos.

Para la primera opción de listar los artículos de la tienda, primero se crea la solicitud a enviar al servidor, esta se envía serializándola entes. Se recibe la respuesta del servidor deserializandola. Se verifica si la respuesta del servidor es la lista de artículos o un solo un mensaje, y de acuerdo con esto se ejecuta la función correspondiente.

```
63          case '2':
64              print("""
65  /*-------------------.
66  | BUSCAR UN ARTÍCULO |
67  `-------------------*/
68  """)
69              buscar = input("Escribe el nombre o la marca del/los artículo(s) a buscar: ").strip()
70
71              solicitud = {"accion": "BUSCAR_ARTICULOS", "buscar": buscar} # Creamos la solicitud como un diccionario
72              cliente.send(json.dumps(solicitud).encode("utf-8")) # Enviamos al servidor la solicitud serializada
73              datosRecibidos = cliente.recv(4096).decode() # Recibe y deserializa los datos recibidos desde el servidor
74
75              resultadoBusqueda = json.loads(datosRecibidos) # Convierte los datos recibidos en JSON a un diccionario
76
77              if ("mensaje" in resultadoBusqueda): # No hubo coincidencias
78                  print()
79                  mostrarMensaje(resultadoBusqueda)
80              else: # Si hubo al menos una coincidencia
81                  mostrarBusqueda(resultadoBusqueda) # # Se muestran los artículos encontrados
82
83              # Esperamos una tecla
84              print("\n>> Presiona una tecla para continuar...")
85              tecla = esperarTecla()
```

Imagen 22. Opción buscar un artículo.

Para la segunda opción de buscar los artículos de la tienda, primero se le solicita al cliente ingresar la marca o el nombre del artículo que quiere buscar, después se crea la solicitud a enviar al servidor, esta se envía serializándola entes. Se recibe la respuesta del servidor deserializandola. Se verifica si la respuesta del servidor es la lista de los artículos encontrados o un solo un mensaje, y de acuerdo con esto se ejecuta la función correspondiente.

```
87          case '3':
88              # Menú para el carrito
89              while (True):
90                  limpiarTerminal()
91                  print("""
92  /*-------------------.
93  | CARRITO DE COMPRAS |
94  `-------------------*/
95
96  >> Elije una de las opciones
97
98  1.- Ver carrito
99  2.- Agregar un artículo al carrito
100 3.- Eliminar un artículo del carrito
101 4.- Volver
102 """)
103
104                 opcion = input("Opción: ").strip()
105
106                 match (opcion):
```

Imagen 23. Submenú carrito de compras.

Para la tercera opción se tiene el submenú del carrito con 4 opciones más: Ver carrito, agregar un artículo al carrito, eliminar un artículo del carrito y volver al menú principal. Y se usa la misma lógica del menú principal.

```
107                 case '1':
108                     print("""
109 /*-----------------------.
110 | ARTÍCULOS EN EL CARRITO |
111 `-----------------------*/
112 """)
113                     solicitud = {"accion": "MOSTRAR_CARRITO"} # Creamos la solicitud como un diccionario
114                     cliente.send(json.dumps(solicitud).encode("utf-8")) # Enviamos al servidor la solicitud serializada
115                     datosRecibidos = cliente.recv(4096).decode() # Recibe y deserializa los datos recibidos desde el servidor
116
117                     articulosCarrito = json.loads(datosRecibidos) # Convierte los datos recibidos en JSON a un diccionario
118
119                     if ("mensaje" in articulosCarrito): # El carrito esta vacío
120                         mostrarMensaje(articulosCarrito)
121                     else: # Si hay al menos un articulo en el carrito
122                         mostrarCarrito(articulosCarrito)
123
124                     # Esperamos una tecla
125                     print("\n>> Presiona una tecla para continuar...")
126                     tecla = esperarTecla()
```

Imagen 24. Opción mostrar carrito.

Para la primera opción de ver el carrito se crea la solicitud a enviar al servidor, esta se envía serializándola entes. Se recibe la respuesta del servidor deserializandola. Se verifica si la respuesta del servidor es la lista de los artículos del carrito o un solo un mensaje, y de acuerdo con esto se ejecuta la función correspondiente.

```
128                     case '2':
129                         print("""
130 /*----------------.
131 | AGREGAR ARTÍCULO |
132 `----------------*/
133 """)
134                         agregar = input("Escribe el id o el nombre del artículo a agregar: ").strip()
135                         cantidad = input("Escribe la cantidad a agregar de ese artículo (máx 5, min 1): ").strip()
136
137                         if (cantidad.isdigit() and (5 >= int(cantidad) > 0)):
138                             solicitud = {"accion": "AGREGAR_CARRITO", "articulo": agregar, "cantidad": cantidad} # Creamos la solicitud como un diccionario
139                             cliente.send(json.dumps(solicitud).encode("utf-8")) # Enviamos al servidor la solicitud serializada
140
141                             datosRecibidos = cliente.recv(4096).decode() # Recibe y deserializa los datos recibidos desde el servidor
142                             confirmacion = json.loads(datosRecibidos) # Convierte los datos recibidos en JSON a un diccionario
143
144                             if ("mensaje" in confirmacion): # El carrito esta vacío
145                                 print()
146                                 mostrarMensaje(confirmacion)
147                         else:
148                             print("\n>> Ingresa una cantidad válida")
149
150                         # Esperamos una tecla
151                         print("\n>> Presiona una tecla para continuar...")
152                         tecla = esperarTecla()
```

Imagen 25. Opción agregar artículo al carrito.

Para la segunda opción de agregar un artículo al carrito, primero se le solicita al cliente ingresar el id o el nombre del artículo que quiere agregar, así como la cantidad de unidades de ese artículo, después se crea la solicitud a enviar al servidor, esta se envía serializándola entes. Se recibe la respuesta del servidor deserializandola. Se verifica si la respuesta del servidor es el mensaje de confirmación y se ejecuta la función correspondiente.

```
154                     case '3':
155                         print("""
156 /*-----------------.
157 | ELIMINAR ARTÍCULO |
158 `-----------------*/
159 """)
160                         eliminar = input("Escribe el id o el nombre del artículo a eliminar: ").strip()
161                         cantidad = input("Escribe la cantidad a eliminar de ese artículo: ").strip()
162
163                         if (cantidad.isdigit() and int(cantidad) > 0):
164                             solicitud = {"accion": "ELIMINAR_CARRITO", "articulo": eliminar, "cantidad": cantidad} # Creamos la solicitud como un diccionario
165                             cliente.send(json.dumps(solicitud).encode("utf-8")) # Enviamos al servidor la solicitud serializada
166
167                             datosRecibidos = cliente.recv(4096).decode() # Recibe y deserializa los datos recibidos desde el servidor
168                             confirmacion = json.loads(datosRecibidos) # Convierte los datos recibidos en JSON a un diccionario
169
170                             if ("mensaje" in confirmacion): # El carrito esta vacío
171                                 print()
172                                 mostrarMensaje(confirmacion)
173                         else:
174                             print("\n>> Ingresa una cantidad válida")
175
176                         # Esperamos una tecla
177                         print("\n>> Presiona una tecla para continuar...")
178                         tecla = esperarTecla()
```

Imagen 26. Opción eliminar artículo del carrito.

Para la tercera opción de eliminar un artículo del carrito, primero se le solicita al cliente ingresar el id o el nombre del artículo que quiere eliminar, así como la cantidad de unidades de ese artículo, después se crea la solicitud a enviar al servidor, esta se envía serializándola entes. Se recibe la respuesta del servidor deserializandola. Se verifica si la respuesta del servidor es el mensaje de confirmación y se ejecuta la función correspondiente.

```
190         case '4':
191             print("""
192 /*-------------------.
193 | FINALIZAR LA COMPRA |
194 `-------------------*/
195 """)
196             solicitud = {"accion": "FINALIZAR_COMPRA"} # Creamos la solicitud como un diccionario
197             cliente.send(json.dumps(solicitud).encode("utf-8")) # Enviamos al servidor la solicitud serializada
198             datosRecibidos = cliente.recv(4096).decode() # Recibe y deserializa los datos recibidos desde el servidor
199 
200             totalPagar = json.loads(datosRecibidos) # Convierte los datos recibidos en JSON a un diccionario
201 
202             if ("mensaje" in totalPagar):
203                 mostrarMensaje(totalPagar)
204 
205             # Esperamos una tecla
206             print("\n>> Presiona una tecla para continuar...")
207             tecla = esperarTecla()
208 
209         case '5':
210             print("\n>> Gracias por visitar nuestra tienda, esperamos volverte a ver")
211             cliente.close()
212             break
```

Imagen 27. Opción finalizar compra.

Para la cuarta opción de finalizar la compra primero se crea la solicitud a enviar al servidor, esta se envía serializándola entes. Se recibe la respuesta del servidor deserializandola. Se verifica si la respuesta del servidor es el mensaje de confirmación y de acuerdo con esto se ejecuta la función correspondiente.

Y para la quinta opción, solo se cierra el socket del cliente.

4.- Archivo: ServidorTienda.py

```
14 import socket, json
15 from FuncionesServidor import obtenerRuta, enviarArticulos, buscarArticulo
16 from FuncionesServidor import agregarCarrito, eliminarCarrito, finalizarCompra
17 
18 # Obetenos las rutas de los archivos JSON (Articulos.json, Carrito.json) y de la carpeta del script
19 carpetaScript, rutaArticulos, rutaCarrito = obtenerRuta()
20 
21 # Cremos el servisor y lo conectamos al cliente
22 servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # Creamos al servidor (dirección IPv4, protocolo TCP)
23 servidor.bind(("127.0.0.1", 5000)) # Conectamos el servidor a la IP y al puerto
24 servidor.listen(2) # El socket esta en modo servidor, (2) = espera una solicitud en cola
25 
26 print(">> Servidor en espera de clientes...")
```

Imagen 28. Creación del socket del servidor.

Líneas [14-16]: Se importan los módulos y las funciones del archivo '**FuncionesServidor.py**'.

Línea [19]: Se obtiene la ruta de la carpeta donde se ejecuta el archivo, así como de los archivos **'Articulos.json'** y **'Carrito.json'**, llamando a la función **obtenerRuta()**.

Líneas [22-24]: Se crea el socket del servidor, e indicamos que usaremos direcciones IPv4 y el protocolo TCP. Conectamos el socket del servidor a la dirección **"127.0.0.1"** y al puerto 5000. Y por último ponemos al socket en modo servidor.

```
28  # Aquí empieza el servidor a operar
29  while (True): # El servidor simpre activo
30      conn, addr = servidor.accept() # Espeta que un cliente se conecte al sock
31
32      print(f"\n>> Cliente conectado desde: {addr}")
33
34      while (True): # Se mantiene la conexión abierta con el cliente
35          # Verificamos la conexión con el cliente
36          try:
37              pedido = conn.recv(4096).decode() # Recibimos la solicitud del cl
38
39              # Verificamos si la solicitud llego
40              if not pedido:
41                  print("\n>> Cliente desconectado")
42
43                  break
44
45              print(f"\n>> Se recibe desde el cliente la solicitud: {pedido}")
46
47              # Intentamos interpretar la solicitud del cliente (JSON)
48              try:
49                  solicitud = json.loads(pedido) # Convierte la solicitud en JS
50              except json.JSONDecodeError:
51                  conn.send(b"Solicitud no valida")
52
53                  continue
```

```
55              # Revisamos las solicitudes
56              if (solicitud["accion"] == "LISTAR_ARTICULOS"):
57                  enviarArticulos(rutaArticulos, conn)
58              elif (solicitud["accion"] == "BUSCAR_ARTICULOS"):
59                  buscarArticulo(rutaArticulos, solicitud["buscar"], conn)
60              elif (solicitud["accion"] == "MOSTRAR_CARRITO"):
61                  enviarArticulos(rutaCarrito, conn)
62              elif (solicitud["accion"] == "AGREGAR_CARRITO"):
63                  agregarCarrito(rutaArticulos, rutaCarrito, solicitud["articulo"], int(solicitud["cantidad"]), conn)
64              elif (solicitud["accion"] == "ELIMINAR_CARRITO"):
65                  eliminarCarrito(rutaCarrito, solicitud["articulo"], int(solicitud["cantidad"]), conn)
66              elif (solicitud["accion"] == "FINALIZAR_COMPRA"):
67                  finalizarCompra(rutaArticulos, rutaCarrito, conn, carpetaScript)
68              else:
69                  conn.send(b"Comando no reconocido")
70
71          except ConnectionResetError:
72              print("\n>> El cliente cerró la conexión abruptamente")
73
74              break
75
76      conn.close() # Cerrar la conexión con el cliente
```

Imagen 29. Flujo del servidor.

Línea [29]: Se crea un bucle while para que el servidor siempre este activo.

Línea [30]: Se obtiene la conexión con el cliente y la dirección de este.

Línea [34]: Se crea un segundo bucle while para que se mantenga la conexión abierta con el mismo cliente, hasta que este se desconecte.

Línea [34-74]: Se trata de verificar la conexión con el cliente. Si la conexión se interrumpe abruptamente se lanza la excepción **'ConnectionResetError'**, saliéndonos del segundo while y cerrando la conexión con el cliente.

Líneas [36-43]: Se reciben solicitudes del cliente constantemente. Si se llega al punto de no recibir solicitudes, entonces eso quiere decir que el cliente se desconectó, por lo que nos salimos del segundo while, así cerrando la conexión con el cliente.

Líneas [48-53]: Se trata de interpretar la solicitud del cliente (deserializamos la solicitud del cliente), si el cliente no envía una solicitud en formato JSON al momento de serializarla, entonces se lanza la excepción **'json.JSONDecodeError'**, por lo que la solicitud no es válida.

Líneas [56-69]: Se revisa la solicitud del cliente, y vemos si la solicitud coincide con las determinadas. Si coincide con una de ellas, entonces se llama a la función correspondiente, de lo contrario la solicitud no se conoce.

Archivo .txt

1.- Archivo: recibo[fecha actual].txt

```
************ RECIBO DE COMPRA ************

Escuela Superior de Cómputo a 2025-09-28_12-01-18

Galletas Marías (Gamesa)
- Cantidad: 2
- Precio unitario: $28 MXN
- Subtotal: $56 MXN

Chocolates Abuelita (La Abuelita)
- Cantidad: 2
- Precio unitario: $55 MXN
- Subtotal: $110 MXN

Jabón de tocador (Zote)
- Cantidad: 3
- Precio unitario: $18 MXN
- Subtotal: $54 MXN

Shampoo (Sedal)
- Cantidad: 5
- Precio unitario: $68 MXN
- Subtotal: $340 MXN

Pasta dental (Colgate)
- Cantidad: 4
- Precio unitario: $35 MXN
- Subtotal: $140 MXN

Crema humectante (CeraVe)
- Cantidad: 5
- Precio unitario: $95 MXN
- Subtotal: $475 MXN

========================================
TOTAL A PAGAR: $1175 MXN
========================================
```

Imagen.

En este archivo, se muestra el recibo de compra en donde se muestran los artículos comprados con todas sus características, así como el subtotal para cada uno y al final el total a pagar por todo.

CONCLUSIÓN

- García Escamilla Bryan Alexis

En esta práctica las únicas dificultades que se tuvieron fueron con respecto a la lectura y escritura de archivos usando Python, específicamente de archivos JSON. Lo más complicado de entender fue la conversión de objetos Python a objetos JSON y viceversa para poder leer/modificar los archivos JSON. Cabe destacar que para poder hacer todas estas operaciones se usó la biblioteca 'json' de Python que facilitó mucho dichas operaciones.

Por otra parte, la interacción entre el cliente y el servidor fue muy sencillo de comprender ya que la biblioteca 'socket' a pesar de tener las mismas funciones que Java, en Python resultan ser más sencillas de manejar.

- Meléndez Macedonio Rodrigo

Tras realizar esta práctica, ahora tengo un mejor entendimiento de cómo funcionan y se comportan los sockets bloqueantes, pues gracias a la teoría vista en clase y al uso del módulo de Python para crear sockets es que logramos hacer una tienda funcional mediante el uso de sockets bloqueantes y la relación cliente/servidor de estos. En nuestro caso decidimos utilizar el lenguaje Python para realizar esta práctica, pues nos dimos cuenta de que el módulo "socket" de Python comparte similitudes con las bibliotecas vistas en Java, solo que con una estructura más simplificada y fácil de entender.

Al inicio me costó un poco entender cómo es que el cliente iba a interactuar con la tienda (servidor), pues el único inconveniente con el que tuve que lidiar al momento de realizar la práctica fue la de manipular los archivos "json" para extraer la información de los artículos de la tienda, así como la de los artículos del carrito seleccionados por el cliente. Fuera de eso, la práctica, aunque un poco extensa en cuanto a contenido, fue fácil de desarrollar ya que utilice mis conocimientos previos que tenía en Python.